

New issue



[Bug]: The performance for Prefix Caching is very unstable for different requests !!!! #3918

Closed

Labels

bug



DefTruth opened on Apr 8, 2024

Contributor



Your current environment

PyTorch version: 2.1.2+cu121

Is debug build: False

CUDA used to build PyTorch: 12.1

ROCM used to build PyTorch: N/A

OS: Ubuntu 22.04.3 LTS (x86_64)

GCC version: (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0

Clang version: Could not collect

CMake version: version 3.27.6

Libc version: glibc-2.35

Python version: 3.10.12 (main, Jun 11 2023, 05:26:28) [GCC 11.4.0] (64-bit runtime)

Python platform: Linux-4.18.0-240.el8.x86_64-x86_64-with-glibc2.35

Is CUDA available: True

CUDA runtime version: 12.2.140

CUDA_MODULE_LOADING set to: LAZY

GPU models and configuration:

GPU 0: NVIDIA L20

GPU 1: NVIDIA L20

GPU 2: NVIDIA L20

GPU 3: NVIDIA L20

GPU 4: NVIDIA L20

GPU 5: NVIDIA L20

GPU 6: NVIDIA L20

GPU 7: NVIDIA L20

Nvidia driver version: 550.54.15

cuDNN version: Probably one of the following:

```
/usr/lib/x86_64-linux-gnu/libcudnn.so.8.9.5
/usr/lib/x86_64-linux-gnu/libcudnn_adv_infer.so.8.9.5
/usr/lib/x86_64-linux-gnu/libcudnn_adv_train.so.8.9.5
/usr/lib/x86_64-linux-gnu/libcudnn_cnn_infer.so.8.9.5
/usr/lib/x86_64-linux-gnu/libcudnn_cnn_train.so.8.9.5
/usr/lib/x86_64-linux-gnu/libcudnn_ops_infer.so.8.9.5
/usr/lib/x86_64-linux-gnu/libcudnn_ops_train.so.8.9.5
```

HIP runtime version: N/A

MIOpen runtime version: N/A

Is XNNPACK available: True

CPU:

```
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:         52 bits physical, 57 bits virtual
Byte Order:            Little Endian
CPU(s):                128
On-line CPU(s) list:   0-127
Vendor ID:             GenuineIntel
BIOS Vendor ID:        Intel(R) Corporation
Model name:            Intel(R) Xeon(R) Gold 6430
BIOS Model name:       Intel(R) Xeon(R) Gold 6430
CPU family:            6
Model:                 143
Thread(s) per core:    2
Core(s) per socket:    32
Socket(s):             2
Stepping:              8
Frequency boost:        enabled
CPU max MHz:           2101.0000
CPU min MHz:           800.0000
BogoMIPS:              4200.00
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca
cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp
lm constant_tsc art arch_perfmon pebs bts rep_good nopl xtopology nonstop_tsc cpuid
aperfmonperf tsc_known_freq pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3 sdbg
fma cx16 xtpr pdcm pcid dca sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave
avx f16c rdrand lahf_lm abm 3dnowprefetch cpuid_fault epb cat_l3 cat_l2 cdp_l3
invpcid_single cdp_l2 ssbd mba ibrs ibpb stibp ibrs_enhanced tpr_shadow vnmi flexpriority
ept vpid ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid cqm rdt_a avx512f
avx512dq rdseed adx smap avx512ifma clflushopt clwb intel_pt avx512cd sha_ni avx512bw
avx512vl xsaveopt xsavec xgetbv1 xsaves cqm_llc cqm_occup_llc cqm_mbm_total cqm_mbm_local
avx512_bf16 wbnoinvd dtherm ida arat pln pts hwp hwp_act_window hwp_epp hwp_pkg_req
avx512vbmi umip pku ospke waitpkg avx512_vbmi2 gfni vaes vpclmulqdq avx512_vnni
avx512_bitalg tme avx512_vpopcntdq la57 rdpid cldemote movdiri movdir64b md_clear pconfig
flush_l1d arch_capabilities
Virtualization:         VT-x
L1d cache:             3 MiB (64 instances)
L1i cache:             2 MiB (64 instances)
L2 cache:              128 MiB (64 instances)
L3 cache:              120 MiB (2 instances)
NUMA node(s):          2
NUMA node0 CPU(s):     0-31,64-95
NUMA node1 CPU(s):     32-63,96-127
Vulnerability Itlb multihit: Not affected
Vulnerability L1tf:     Not affected
```

Vulnerability Mds: Not affected
 Vulnerability Meltdown: Not affected
 Vulnerability Spec store bypass: Vulnerable
 Vulnerability Spectre v1: Vulnerable: __user pointer sanitization and usercopy barriers only; no swapgs barriers
 Vulnerability Spectre v2: Vulnerable, IBPB: disabled, STIBP: disabled
 Vulnerability Srbds: Not affected
 Vulnerability Tsx async abort: Not affected

Versions of relevant libraries:

[pip3] mypy==0.991
 [pip3] mypy-extensions==1.0.0
 [pip3] numpy==1.22.2
 [pip3] onnx==1.14.0
 [pip3] pytorch-quantization==2.1.2
 [pip3] torch==2.1.2
 [pip3] torch-tensorrt==0.0.0
 [pip3] triton==2.1.0+e621604
 [conda] Could not collect ROCm Version: Could not collect

Neuron SDK Version: N/A

vLLM Version: 0.4.0.post1

vLLM Build Flags:

CUDA Archs: 5.2 6.0 6.1 7.0 7.2 7.5 8.0 8.6 8.7 9.0+PTX; ROCm: Disabled; Neuron: Disabled

GPU Topology:

GPU0	GPU1	GPU2	GPU3	GPU4	GPU5	GPU6	GPU7	CPU Affinity	NUMA		
GPU0	X	PIX	PIX	PIX	SYS	SYS	SYS	SYS	0-31,64-95	0	N/A
GPU1	PIX	X	PIX	PIX	SYS	SYS	SYS	SYS	0-31,64-95	0	N/A
GPU2	PIX	PIX	X	PIX	SYS	SYS	SYS	SYS	0-31,64-95	0	N/A
GPU3	PIX	PIX	PIX	X	SYS	SYS	SYS	SYS	0-31,64-95	0	N/A
GPU4	SYS	SYS	SYS	SYS	X	PIX	PIX	PIX	32-63,96-127	1	N/A
GPU5	SYS	SYS	SYS	SYS	PIX	X	PIX	PIX	32-63,96-127	1	N/A
GPU6	SYS	SYS	SYS	SYS	PIX	PIX	X	PIX	32-63,96-127	1	N/A
GPU7	SYS	SYS	SYS	SYS	PIX	PIX	PIX	X	32-63,96-127	1	N/A

Legend:

X = Self
 SYS = Connection traversing PCIe as well as the SMP interconnect between NUMA nodes (e.g., QPI/UPI)
 NODE = Connection traversing PCIe as well as the interconnect between PCIe Host Bridges within a NUMA node
 PHB = Connection traversing PCIe as well as a PCIe Host Bridge (typically the CPU)
 PXB = Connection traversing multiple PCIe bridges (without traversing the PCIe Host Bridge)
 PIX = Connection traversing at most a single PCIe bridge
 NV# = Connection traversing a bonded set of # NVLinks

Describe the bug

The first token performance for **Prefix Caching** is very un-stable for different random requests, in my case, range from 50ms -> 500ms, and the mean time of first token vs **No Prefix Caching** is: 337ms vs 390ms, only 60ms boost.

```
[I] [1/20] 5.00% output_len: 484, first_token: 58.25 ms, request_latency: 25.75 s
[I] [2/20] 10.00% output_len: 493, first_token: 88.23 ms, request_latency: 26.22 s
[I] [3/20] 15.00% output_len: 489, first_token: 468.82 ms, request_latency: 26.49 s
[I] [4/20] 20.00% output_len: 480, first_token: 348.85 ms, request_latency: 26.23 s
[I] [5/20] 25.00% output_len: 499, first_token: 470.80 ms, request_latency: 26.09 s
[I] [6/20] 30.00% output_len: 499, first_token: 430.00 ms, request_latency: 25.74 s
[I] [7/20] 35.00% output_len: 488, first_token: 432.71 ms, request_latency: 25.43 s
[I] [8/20] 40.00% output_len: 495, first_token: 481.83 ms, request_latency: 25.06 s
[I] [9/20] 45.00% output_len: 475, first_token: 495.65 ms, request_latency: 24.66 s
[I] [10/20] 50.00% output_len: 494, first_token: 494.50 ms, request_latency: 24.21 s
[I] [11/20] 55.00% output_len: 500, first_token: 70.87 ms, request_latency: 23.76 s
[I] [12/20] 60.00% output_len: 497, first_token: 255.73 ms, request_latency: 23.71 s
[I] [13/20] 65.00% output_len: 500, first_token: 455.26 ms, request_latency: 23.44 s
[I] [14/20] 70.00% output_len: 500, first_token: 294.82 ms, request_latency: 22.94 s
[I] [15/20] 75.00% output_len: 500, first_token: 285.17 ms, request_latency: 22.59 s
[I] [16/20] 80.00% output_len: 482, first_token: 450.05 ms, request_latency: 22.26 s
[I] [17/20] 85.00% output_len: 489, first_token: 59.68 ms, request_latency: 21.71 s
[I] [18/20] 90.00% output_len: 466, first_token: 686.31 ms, request_latency: 21.44 s
[I] [19/20] 95.00% output_len: 498, first_token: 76.72 ms, request_latency: 20.70 s
[I] [20/20] 100.00% output_len: 491, first_token: 344.26 ms, request_latency: 20.34 s
```



DefTruth added bug on Apr 8, 2024



DefTruth on Apr 8, 2024 · edited by DefTruth

Edits ▾

Contributor

Author



some case will range from 70ms -> 900ms

```
[I] [1/20] 5.00% first_token: 409.19 ms
[I] [2/20] 10.00% first_token: 79.29 ms
[I] [3/20] 15.00% first_token: 321.46 ms
[I] [4/20] 20.00% first_token: 605.53 ms
[I] [5/20] 25.00% first_token: 601.83 ms
[I] [6/20] 30.00% first_token: 932.33 ms
[I] [7/20] 35.00% first_token: 290.56 ms
[I] [8/20] 40.00% first_token: 572.87 ms
[I] [9/20] 45.00% first_token: 299.98 ms
[I] [10/20] 50.00% first_token: 595.04 ms
[I] [11/20] 55.00% first_token: 72.29 ms
[I] [12/20] 60.00% first_token: 482.50 ms
[I] [13/20] 65.00% first_token: 338.41 ms
```



DefTruth on Apr 8, 2024

Contributor

Author



[@youkaichao](#) [@WoosukKwon](#) how can i fix this un-stable performance?

but can run very fast if the reqs has been hit the server. The second time i send the same reqs:

```
[I] [1/20] 5.00% first_token: 55.91 ms
[I] [2/20] 10.00% first_token: 70.44 ms
[I] [3/20] 15.00% first_token: 64.54 ms
[I] [4/20] 20.00% first_token: 62.45 ms
[I] [5/20] 25.00% first_token: 63.78 ms
[I] [6/20] 30.00% first_token: 59.13 ms
[I] [7/20] 35.00% first_token: 59.25 ms
[I] [8/20] 40.00% first_token: 63.24 ms
[I] [9/20] 45.00% first_token: 77.15 ms
[I] [10/20] 50.00% first_token: 57.52 ms
[I] [11/20] 55.00% first_token: 76.91 ms
[I] [12/20] 60.00% first_token: 72.77 ms
[I] [13/20] 65.00% first_token: 71.37 ms
[I] [14/20] 70.00% first_token: 54.13 ms
[I] [15/20] 75.00% first_token: 85.11 ms
[I] [16/20] 80.00% first_token: 76.92 ms
[I] [17/20] 85.00% first_token: 73.56 ms
[I] [18/20] 90.00% first_token: 94.43 ms
[I] [19/20] 95.00% first_token: 68.28 ms
[I] [20/20] 100.00% first_token: 59.77 ms
```



but with random access reqs with some same system prompt, vllm server still run into very un-stable performance:

```
[I] [1/20] 5.00% first_token: 409.19 ms
[I] [2/20] 10.00% first_token: 79.29 ms
[I] [3/20] 15.00% first_token: 321.46 ms
[I] [4/20] 20.00% first_token: 605.53 ms
[I] [5/20] 25.00% first_token: 601.83 ms
[I] [6/20] 30.00% first_token: 932.33 ms
[I] [7/20] 35.00% first_token: 290.56 ms
[I] [8/20] 40.00% first_token: 572.87 ms
[I] [9/20] 45.00% first_token: 299.98 ms
[I] [10/20] 50.00% first_token: 595.04 ms
[I] [11/20] 55.00% first_token: 72.29 ms
[I] [12/20] 60.00% first_token: 482.50 ms
[I] [13/20] 65.00% first_token: 338.41 ms
```



Can you share the client / some sample data you are using to make requests so I can reproduce and take a look?



DefTruth on Apr 8, 2024 · edited by DefTruth

Edits ▾

Contributor

Author



@robertgshaw2-neuralmagic

ref:

The issue is that the attention kernel needed for prefix caching is triton-based. The first time this kernel is encountered, it generates code, causing overhead. This is why it occurs on the second request but never again (and I suspect the triton JITs are cached in between server launches as well, which is why it only occurs once per Docker container)

in my case:

prompt = system prompt (static) + user_prompt (dyn)

will each reqs with a specific prompt len cause the triton-based kernel re-gen every time?



robertgshaw2-redhat on Apr 8, 2024

Sponsor

Collaborator



How varied is the user prompt in terms of length? In general, it is not surprising to see variation in TTFT (e.g. if you have some prompts with 2000 tokens and some with 10 tokens, you will see quite a bit of variation).

Can you share some sample data you are sending up to the server (and the client you are using)? I can try to reproduce your results



DefTruth on Apr 9, 2024 · edited by DefTruth

Edits ▾

Contributor

Author



@robertgshaw2-neuralmagic I am sorry, I can not provide the private data here, but some logging message are available.

- The second time I send 10 same requests to the vLLM server, the run very fast for first token, QPS=1.

⚠ Caution

However, in a real production environment, we don't send the user's request a second time, only once

```
-----
[I] Total selected input requests count: 10, random: False, prefix caching: True
[I] [ 1/10    10.00%] output_len: 80 , prompt_len: 1332, first_token: 79.02 ms, other
[I] [ 2/10    20.00%] output_len: 114 , prompt_len: 1378, first_token: 61.02 ms, other
[I] [ 3/10    30.00%] output_len: 58 , prompt_len: 1386, first_token: 56.80 ms, other
[I] [ 4/10    40.00%] output_len: 92 , prompt_len: 753 , first_token: 49.17 ms, other
```



```

[I] [ 5/10 50.00%] output_len: 10 , prompt_len: 1116, first_token: 72.12 ms, other
[I] [ 6/10 60.00%] output_len: 56 , prompt_len: 812 , first_token: 83.49 ms, other
[I] [ 7/10 70.00%] output_len: 126 , prompt_len: 1141, first_token: 67.73 ms, other
[I] [ 8/10 80.00%] output_len: 43 , prompt_len: 931 , first_token: 48.25 ms, other
[I] [ 9/10 90.00%] output_len: 47 , prompt_len: 1208, first_token: 77.25 ms, other
[I] [ 10/10 100.00%] output_len: 120 , prompt_len: 1212, first_token: 61.48 ms, other
-----

```

- With random selected requests(many of them has not been send to vLLM server), the first token performance run very un-stable, range from 60ms to 500ms, the prompt len of these requests locate in a small range 800-1200 (vs 102000), the static system prompt len of each request are 216, QPS=1.

```

-----
[I] Total selected input requests count: 10, random: True, prefix caching: True
[I] [ 1/10 10.00%] output_len: 26 , prompt_len: 1485, first_token: 61.76 ms, other
[I] [ 2/10 20.00%] output_len: 71 , prompt_len: 1996, first_token: 563.94 ms, other
[I] [ 3/10 30.00%] output_len: 35 , prompt_len: 1886, first_token: 178.58 ms, other
[I] [ 4/10 40.00%] output_len: 115 , prompt_len: 1746, first_token: 482.35 ms, other
[I] [ 5/10 50.00%] output_len: 63 , prompt_len: 1776, first_token: 140.30 ms, other
[I] [ 6/10 60.00%] output_len: 24 , prompt_len: 1864, first_token: 489.39 ms, other
[I] [ 7/10 70.00%] output_len: 104 , prompt_len: 1755, first_token: 476.24 ms, other
[I] [ 8/10 80.00%] output_len: 24 , prompt_len: 850 , first_token: 245.66 ms, other
[I] [ 9/10 90.00%] output_len: 118 , prompt_len: 1401, first_token: 426.03 ms, other
[I] [ 10/10 100.00%] output_len: 320 , prompt_len: 1924, first_token: 337.66 ms, other
-----

```

- Without Prefix Caching , each requests run very stable without random or not, QPS=1.

```

-----
[I] Total selected input requests count: 10, random: True, prefix caching: False
[I] [ 1/10 10.00%] output_len: 9 , prompt_len: 1232, first_token: 397.72 ms, other
[I] [ 2/10 20.00%] output_len: 63 , prompt_len: 845 , first_token: 302.29 ms, other
[I] [ 3/10 30.00%] output_len: 14 , prompt_len: 961 , first_token: 308.66 ms, other
[I] [ 4/10 40.00%] output_len: 49 , prompt_len: 1606, first_token: 494.45 ms, other
[I] [ 5/10 50.00%] output_len: 145 , prompt_len: 1271, first_token: 411.29 ms, other
[I] [ 6/10 60.00%] output_len: 117 , prompt_len: 2012, first_token: 603.70 ms, other
[I] [ 7/10 70.00%] output_len: 36 , prompt_len: 1372, first_token: 441.37 ms, other
[I] [ 8/10 80.00%] output_len: 67 , prompt_len: 1497, first_token: 461.37 ms, other
[I] [ 9/10 90.00%] output_len: 96 , prompt_len: 1183, first_token: 407.04 ms, other
[I] [ 10/10 100.00%] output_len: 225 , prompt_len: 1391, first_token: 425.24 ms, other
-----
[I] Total selected input requests count: 10, random: False, prefix caching: False
[I] [ 1/10 10.00%] output_len: 71 , prompt_len: 1378, first_token: 426.79 ms, other
[I] [ 2/10 20.00%] output_len: 79 , prompt_len: 1332, first_token: 420.03 ms, other
[I] [ 3/10 30.00%] output_len: 10 , prompt_len: 1116, first_token: 347.30 ms, other
[I] [ 4/10 40.00%] output_len: 64 , prompt_len: 1386, first_token: 421.31 ms, other
[I] [ 5/10 50.00%] output_len: 98 , prompt_len: 753 , first_token: 250.06 ms, other
[I] [ 6/10 60.00%] output_len: 52 , prompt_len: 812 , first_token: 288.06 ms, other
[I] [ 7/10 70.00%] output_len: 118 , prompt_len: 1141, first_token: 353.77 ms, other
[I] [ 8/10 80.00%] output_len: 53 , prompt_len: 931 , first_token: 316.57 ms, other
-----

```

```
[I] [ 9/10 90.00%] output_len: 47 , prompt_len: 1208, first_token: 393.68 ms, other
[I] [ 10/10 100.00%] output_len: 120 , prompt_len: 1212, first_token: 409.33 ms, other
-----
```



DefTruth on Apr 9, 2024

Contributor

Author



closed, I found that when there are enough requests, there will still be performance improvements, although unstable.



DefTruth closed this as completed on Apr 9, 2024

[Sign up for free](#) to join this conversation on **GitHub**. Already have an account? [Sign in to comment](#)

Metadata

Assignees

No one assigned

Labels

bug

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Participants

